

Передача проекта в Claude Code

Этот файл — вместе с FOOTBALL_GAME_MASTER_CONTEXT.md и football_prototype_0_v5.jsx — всё, что нужно новой сессии Claude Code, чтобы продолжить с того места, где мы остановились в чате.

Что уже сделано и проверено

Прототип 0 прошёл 5 итераций в виде интерактивного React-артефакта прямо в чате и подтвердил рабочий игровой цикл:

- создание игрока в 6 шагов (имя/фамилия → возраст → страна → позиция → профиль → сложность), нигде не показывается OVR или potential
- Player Generator: позиция + профиль + сложность + случайность → характеристики → OVR (всегда производный, никогда не задаётся напрямую)
- скрытый potential, влияющий на скорость развития
- календарь с одним действием в игровой день (тренировка/восстановление/отдых), дата реально продвигается, в день матча подготовка недоступна
- накопительный прогресс характеристик (0→100 за характеристику, не прыжки по +1)
- position-aware Match Engine: 12 конкретных шаблонов момента, сгруппированных под 5 концептуальных групп (фланг / один на один / стандарт / контратака / защитный эпизод), отфильтрованных по 5 группам позиций (GK/DEF/MID/WING/ATT)
- цепочки моментов (решение влияет на следующую ситуацию), последствия, LIVE-хроника матча с фоновыми событиями
- послематчевый отчёт с прогресс-барами развития

Это рабочее ядро. Задача Claude Code — не начинать заново, а перенести эту логику в нормальную структуру проекта.

Первая задача

НЕ переписывать логику. Собрать нормальный проект (Vite + React + TypeScript) и

перенести в него `football_prototype_0_v5.jsx` один в один, но разложив по модулям в духе архитектуры из Master Context:

```
src/
  engine/      (Football Engine: шаблоны, resolve*, position-aware отбор)
  career/      (Career Engine: генератор игрока, прогресс, календарь)
  ui/          (экраны: создание, home, матч, отчёты)
  data/        (POSITIONS, PROFILES, COUNTRIES, DIFFICULTIES, OPPONENTS)
```

После переноса — показать, что поведение не изменилось: та же логика, просто в файлах вместо одного артефакта.

Известные грабли (чтобы не наступать заново)

- Тексты последствий (`outcomes`) — обычные строки, не функции. Если понадобится вставить `${o.name}` в исход — сначала оберните в `(o) => ({...})` , иначе `ReferenceError` (уже наступали).
- У каждой позиционной группы должно быть ≥ 3 шаблона в пуле — иначе на 3-м слоте матча фильтр «без повторов» вернёт пустой массив и всё упадёт.
- Фраза-заглушка при неудачной обороне должна быть нейтральной («Соперник не реализует момент», не «Вратарь спасает») — вратарь сам иногда защищается, других вратарей в игре нет.
- ОВР пересчитывается после ЛЮБОГО изменения характеристик — и после тренировки, и после матча, не только после матча.
- Сложность и профиль всегда сдвигают вводные параметры генерации, никогда не задают фиксированный ОВР или характеристику напрямую.

С чего начать сессию в Claude Code

Скопируйте это первым сообщением:

```
Прочитай FOOTBALL_GAME_MASTER_CONTEXT.md и PROJECT_HANDOFF.md
полностью. Прочитай football_prototype_0_v5.jsx как референсную реализацию
— это уже согласованная и протестированная логика, не предлагай её менять.
Собери нормальную структуру Vite + React + TypeScript проекта и перенеси эту
логику один в один, без переписывания механик. Покажи, что поведение
совпадает с прототипом, прежде чем предлагать что-то новое.
```

